

EXTREME HEATWAVE EARLY WARNING

&

HUMAN THERMAL STRESS INDEX

Technical Architecture • Software Requirements Specification

ML + Physics + Agent + Backend + Frontend

SIH Prototype · Version 1.0 · September 2026
Decision-support prototype. Not medical advice.

TABLE OF CONTENTS

01	Executive Summary	3
02	Problem Understanding	4
03	End-to-End System Explanation	5
04	Goals and Non-Goals	6
05	Software Requirements Specification	7
06	Functional Requirements	8
07	Non-Functional Requirements	11
08	Architecture	12
09	Domain Ownership	14
10	Data Flow	15
11	Weather System	16
12	Spatial System	17
13	Physics System	18
14	ML / Risk System	19
15	Agent System	20
16	Backend System	22
17	Frontend System	23
18	Database	24
19	API Contracts	25
20	Agent Tool Contracts	27
21	Monorepo Structure	28
22	Scaffold Routes	29
23	Environment Configuration	30
24	Testing Strategy	31

25	Failure Handling	32
26	Observability	33
27	Security & Safety	34
28	24-Hour Execution Plan	35
29	Team Parallelization Plan	36
30	Domain-Specific Agent Prompts	37
31	Demo Flow	39
32	Competitive Differentiation	40
33	Limitations	41
34	Traceability Matrix	42
35	Definition of Done	43

01

Executive Summary

Understanding the transformation from weather to municipal action.

The **Heatwave Early Warning & Human Thermal Stress Index (HEEWS)** system is a decision-support prototype for the city of Hyderabad. It pulls a 3–5 day weather forecast, converts it into per-ward human thermal-stress metrics (WBGT and UTCI), combines those with ward-level vulnerability data, and produces a population-level heat health risk index (MRI) per ward. A reasoning agent then recommends concrete municipal interventions.

It is **not** a clinical mortality predictor. It is a defensible, reproducible, audit-friendly decision-support system that helps a municipal command center decide where, when, and how to act.

Core transformation

Core Transformation

WEATHER → PHYSIOLOGICAL HEAT STRESS → POPULATION RISK → DECISION → ACTION

Every component in the architecture implements exactly one arrow in this chain.

02

Problem Understanding

Why raw temperature warnings are insufficient for heat-health decisions.

Cities like Hyderabad experience extreme heatwaves that cause preventable mortality and morbidity, especially among outdoor workers, the elderly, and residents of informal housing. Most existing systems provide either raw temperature warnings or generic citywide advisories that miss the most vulnerable wards.

Air temperature alone is insufficient because human heat stress depends on humidity, wind, radiation, and pressure. Two days with identical air temperature can produce wildly different physiological outcomes. Ward-level resolution matters because Urban Heat Island intensity and vulnerability composition vary sharply between wards.

03

End-to-End System Explanation

Walkthrough from forecast to municipal action.

Illustrative values for Ward A (clearly labeled as examples):

Variable	Value	Unit
Temperature	41.2	°C
Relative humidity	58	%
Wind speed	2.1	m/s

Shortwave radiation	710	W/m ²
WBGT (Liljegren)	33.8	°C
UTCI	44.6	°C
MRI	82 / 100	—
Band	EXTREME	—

Agent Reasoning (illustrative)

High thermal stress, high outdoor-worker share, persistent multi-day heat. Recommend activating cooling center, issuing outdoor-work advisory, and scheduling welfare checks.

04

Goals and Non-Goals

What this MVP must do, and what it explicitly does not.

#	Goal	Priority
G1	Fetch Open-Meteo forecast for Hyderabad	MUST
G2	Downscale to ward-level weather	MUST
G3	Compute WBGT and UTCI per ward per hour	MUST
G4	Generate 3–5 day forecast heat-health risk per ward	MUST
G5	Produce explainable vulnerability-aware MRI	MUST
G6	Reasoning agent proposes municipal actions	MUST
G7	Render a GIS dashboard of Hyderabad ward risk	MUST
G8	Public-friendly summary view	SHOULD
G9	Run end-to-end on synthetic data offline	MUST
G10	End-to-end demo runs in under 5 minutes	MUST

Non-Goals (out of scope for MVP)

Out of Scope

Clinical mortality prediction · individual diagnosis · real-time telemetry · SMS/push delivery · authenticated workflows · cross-city federation.

05

Software Requirements Specification

Authoritative specification; this chapter summarizes the full document.

This chapter is the SRS core. Detailed schemas are in chapter 19 (API Contracts) and chapter 20 (Agent Tool Contracts). Requirements are tagged MUST, SHOULD, COULD, CUT.

06

Functional Requirements

Numbered FR-### requirements across all domains.

FR-W — Weather Ingestion

ID	Requirement	Priority
FR-W-001	Fetch current weather for Hyderabad bounding box	MUST
FR-W-002	Fetch 3–5 day hourly forecast	MUST
FR-W-003	Normalize units (km/h → m/s, hPa → Pa)	MUST
FR-W-004	Validate presence of required fields	MUST
FR-W-005	Store raw_weather_staging rows with source timestamp	MUST
FR-W-006	Cache last-good response per pipeline_run_id	MUST
FR-W-007	Support deterministic mock fixtures	MUST
FR-W-008	Surface fetch errors with explicit codes	MUST

FR-S — Spatial Processing

ID	Requirement	Priority
FR-S-001	Load ward polygon GeoJSON	MUST
FR-S-002	Validate polygon geometry	MUST
FR-S-003	Assign stable ward_id (UUID)	MUST
FR-S-004	Compute centroid latitude/longitude	MUST
FR-S-005	Run IDW interpolation from grid → centroid	MUST
FR-S-006	Handle missing weather points gracefully	MUST
FR-S-007	Reject duplicate ward_number without unique stable_id	MUST

FR-P — Physics Engine

ID	Requirement	Priority
FR-P-001	Compute solar geometry	MUST
FR-P-002	Compute WBGT using Liljegren formulation	MUST
FR-P-003	Compute simplified WBGT fallback	MUST
FR-P-004	Compute UTCI	MUST
FR-P-005	Validate all physics inputs	MUST
FR-P-006	Emit thermal_stress JSON with explicit units	MUST
FR-P-007	Pin physics_version string per run	MUST
FR-P-008	Reference test cases for known WBGT/UTCI	MUST

FR-R — Risk Model

ID	Requirement	Priority
FR-R-001	Construct thermal features	MUST
FR-R-002	Construct temporal features	MUST
FR-R-003	Construct vulnerability features	MUST
FR-R-004	Compute MRI in [0, 100]	MUST
FR-R-005	Map MRI → band	MUST
FR-R-006	Return top contributing factors	MUST
FR-R-007	Pin risk_model_version string per run	MUST
FR-R-008	Deterministic inference	MUST

FR-A — Agent

ID	Requirement	Priority
FR-A-001	Retrieves risk per ward	MUST
FR-A-002	Retrieves forecast per ward	MUST
FR-A-003	Retrieves vulnerability per ward	MUST
FR-A-004	Retrieves resources	MUST
FR-A-005	Reasons over ranked wards	MUST
FR-A-006	Returns strictly structured JSON	MUST
FR-A-007	Never invents weather or thermal values	MUST
FR-A-008	Never claims mortality certainty	MUST
FR-A-009	Suggests re-check time	MUST
FR-A-010	Falls back to rule-based plan	MUST

FR-B — Backend

ID	Requirement	Priority
FR-B-001	Expose REST API	MUST
FR-B-002	Run pipeline on a schedule	MUST
FR-B-003	Persist all intermediate outputs	MUST
FR-B-004	Serve dashboard frontend	MUST
FR-B-005	Health/version endpoints	MUST
FR-B-006	Stamp pipeline_run_id on responses	MUST
FR-B-007	MOCK and LIVE modes	MUST

FR-F — Frontend

ID	Requirement	Priority
----	-------------	----------

FR-F-001	Render Hyderabad ward choropleth map	MUST
FR-F-002	Color wards by risk_band	MUST
FR-F-003	Ward detail panel	MUST
FR-F-004	3–5 day forecast timeline	MUST
FR-F-005	Agent recommendation panel	MUST
FR-F-006	Alert center	MUST
FR-F-007	Display explanatory reasons	MUST
FR-F-008	Never compute WBGT/UTCI on client	MUST

07

Non-Functional Requirements

MVP bar vs production bar.

Concern	MVP Bar	Production Bar
Performance	p95 ward risk < 500 ms (cached)	< 100 ms with horizontal scaling
Reliability	Ingestion retries 3× with backoff	Multi-region, dead-letter queue
Observability	JSON logs with pipeline_run_id	Distributed tracing, metrics
Explainability	Top-3 contributing factors	Full SHAP / counterfactuals
Reproducibility	Pin physics/risk versions	Immutable model artifacts
Maintainability	Single-command local run	CI/CD, lint, type-check
API consistency	Stable JSON contracts	OpenAPI 3.1 spec published
Fault tolerance	MOCK mode fallback	Circuit breakers
Security	No PII; risk-only outputs	RBAC, audit log
Privacy	Aggregate ward-level only	No PHI ever stored
Scalability	One city	Multi-city federation
Testability	Per-domain unit + integration	90 %+ coverage

08

Architecture

Six orthogonal domains, each owning one responsibility.

The system is split into six orthogonal domains. Each owns a single responsibility and exposes a stable contract. No team may modify another team's internal implementation without changing the contract.

End-to-end architecture

Open-Meteo (LIVE) / fixtures (MOCK)





Arrows in the diagram are contracts. Every contract is a JSON schema defined in chapter 19. Edges are versioned: `physics_version` and `risk_model_version` travel with every payload.

09

Domain Ownership

Clear ownership matrix; no cross-team modifications.

Domain	Owner	Inputs	Outputs	Tech
Weather + Spatial	ML/DS	Open-Meteo + ward GeoJSON	ward_weather	Python
Physics	ML/DS	ward_weather	ward_thermal_stress	Python
Risk	ML/DS	Thermal + vulnerability	ward_mortality_risk	Python/ONNX
Agent	Agent engineer	Risk + context + tools	decision_plan	Python + LLM
Backend	Backend	Domain outputs	REST API, alerts	Go
Frontend	Frontend	Backend API	Dashboard	React

Contract Rule A team may consume another team's outputs but must not modify them. To change a contract, both teams must agree.

10

Data Flow

Step-by-step transformation pipeline.

1. Backend scheduler triggers a pipeline_run_id. 2. Weather domain fetches Open-Meteo. 3. Spatial domain computes centroids and IDW. 4. Physics computes WBGT and UTCI. 5. Risk computes MRI and band. 6. Agent reasons and proposes actions. 7. Backend persists every stage with the run ID. 8. Frontend polls and renders.

11

Weather System

Open-Meteo ingestion and unit normalization.

Open-Meteo is the primary operational input. It is free, no API key, and provides hourly data for a 3–5 day horizon at ~9 km grid resolution.

Required variables: temperature_2m, relative_humidity_2m, wind_speed_10m, shortwave_radiation, direct_radiation, diffuse_radiation, surface_pressure, cloud_cover.

Unit Normalization wind km/h × 0.2778 → m/s. pressure hPa × 100 → Pa. timezone Asia/Kolkata.

12

Spatial System

Ward polygons, centroids, IDW.

ward_id is computed as uuid5(NAMESPACE_URL, f"{ward_name}{ward_number}"). Ward numbers are not unique across zones. The centroid is used to sample Open-Meteo; IDW is the documented fallback when raw grid data become available.

IDW intuition: nearby weather points influence a ward more than distant ones, with weight 1/d². Far-away points fade quickly.

13

Physics System

WBGT (Liljegren) + UTCI + solar geometry.

WBGT blends natural wet-bulb, globe temperature, and shaded dry-bulb. Liljegren uses direct and diffuse radiation, solar zenith, humidity, wind, and pressure. If radiation is missing, the system falls back to a simplified WBGT and emits `wbgt_quality=degraded`.

UTCI is the equivalent air temperature at reference conditions producing the same physiological strain as the actual environment. Inputs: T, RH, v, Tmrt. For MVP we use an operational polynomial approximation pinned by `physics_version`.

14

ML / Risk System

Honest, transparent, vulnerability-aware risk indicator.

No validated mortality ground truth is available to the team. The MRI is therefore a physiologically grounded, vulnerability-aware heat-health risk indicator — NOT a clinical mortality prediction. We state this explicitly in the UI and PDFs.

MRI formula (MVP, transparent):

```
MRI = 100 * (
  0.35 * norm(peak_wbgt_next_72h, 25, 35)
+ 0.20 * norm(utci_now, 26, 46)
+ 0.15 * norm(persistence_days_high, 0, 5)
+ 0.20 * norm(vulnerability_index, 0, 1)
+ 0.10 * norm(-overnight_recovery_gap_c, -10, 0)
)
```

Risk bands:

LOW	0–19
MODERATE	20–39
HIGH	40–59
VERY HIGH	60–79
EXTREME	80–100

15

Agent System

Decision-support and orchestration layer; never replaces science.

What the agent MAY do

Retrieve risk, forecast, vulnerability, resources. Compare wards. Prioritize. Generate explanations, action plans, public advisories. Decide re-check cadence.

What the agent MUST NOT do

Invent weather or thermal values. Calculate WBGT or UTCI itself. Fabricate medical facts or mortality statistics. Override validated outputs. Claim mortality certainty.

Agent tools: `get_ward_risk`, `get_forecast`, `get_thermal_stress`, `get_vulnerability`, `get_resources`, `get_previous_alerts`, `create_alert`, `generate_action_plan`.

Agent loop:

SENSE → PREDICT → ASSESS → REASON → ACT → MONITOR → REASSESS. Each stage is auditable and uses only deterministic tool outputs.

Strict output schema (excerpt):

```
{
  "severity": "EXTREME",
  "priority_wards": ["ward_001", "ward_017"],
  "reasoning_summary": "...",
  "key_factors": ["..."],
  "recommended_actions": [
    { "action": "Activate cooling center", "priority": "HIGH", "target": "ward_001" }
  ],
  "public_advisory": "...",
  "recheck_interval_minutes": 60
}
```

16

Backend System

Go service: HTTP API, persistence, orchestration, alerts.

Method	Route	Purpose	Owner
GET	/api/health	Liveness	Backend
GET	/api/version	Versions	Backend
GET	/api/wards	List wards	Backend
GET	/api/wards/{id}	Ward detail	Backend
GET	/api/wards/{id}/weather	Weather	Backend
GET	/api/wards/{id}/thermal	Thermal stress	Backend
GET	/api/wards/{id}/risk	MRI + band	Backend
GET	/api/forecast	Citywide forecast	Backend
GET	/api/alerts	Active alerts	Backend
POST	/api/agent/action-plan	Agent recommendation	Backend + Agent
POST	/api/pipeline/run	Trigger full run	Backend

17

Frontend System

GIS map, ward detail, forecast timeline, alerts, decision panel.

Five core screens:

1. City Overview — choropleth map of wards. (3. Ward Detail — T, RH, v, rad, WBGT, UTCI, MRI, band. 3. Forecast Timeline — 3–5 day risk evolution. 4. Alert Center — alerts with status. 5. Decision Panel — agent explanation + actions.

Frontend Constraint Never compute WBGT/UTCI on the browser. Display physics_version and risk_model_version. Show the disclaimer banner at all times.

18

Database

ward_id is the stable UUID join key.

Table	Purpose
ward_boundaries	Polygon GeoJSON per ward
ward_demographics	Elderly, outdoor workers, housing, population
raw_weather_staging	Raw Open-Meteo payloads
ward_weather	Per-ward per-timestamp normalized weather
ward_thermal_stress	WBGT + UTCI per ward per timestamp
ward_mortality_risk	MRI + band + top factors per ward per run
cooling_centers	Resources per ward
hospital_capacity	Beds available per ward
alerts	Persisted alerts
pipeline_runs	Per-run stages and latency
agent_decisions	Agent plan per run per ward

19

API Contracts

JSON schemas between every domain.

All endpoints return JSON. All responses include pipeline_run_id, physics_version, risk_model_version where applicable. Error envelope is documented in the SRS.

Example: GET /api/wards/{id}/risk

```
{
  "ward_id": "ward_001",
  "risk": { "score": 82, "band": "EXTREME",
            "top_factors": ["Peak WBGT = 34.7 °C",
                           "UTCI now = 45.1 °C"] },
  "thermal": { "wbgt_c": 33.8, "utci_c": 44.6 },
  "model_version": "risk-v1.0.0"
}
```

Agent Tool Contracts

Tool wrappers and strict output schema.

Tool	Purpose
get_ward_risk	Retrieve MRI + band + top factors
get_forecast	Retrieve forecast
get_thermal_stress	Retrieve WBGT, UTCI
get_vulnerability	Retrieve vulnerability features
get_resources	Retrieve cooling centers, hospitals
get_previous_alerts	Retrieve alert history
create_alert	Persist alert (side-effect)
generate_action_plan	Persist action plan (side-effect)
Failure Behavior	If the LLM fails or times out, use the rule-based fallback plan and set fallback_used=true. The plan is still persisted.

21 Monorepo Structure

Practical scaffold for parallel team work.

```
heatwave-early-warning/  
  docs/           # SRS, ARCHITECTURE, API_CONTRACTS, AGENT_SPEC, ...  
  backend/        # Go service: cmd/, internal/{api, services, ...}  
  ml/             # Python: ingestion, spatial, physics, risk  
  agent/          # Python: tools, prompts, schemas, workflows  
  frontend/       # React + Vite + Leaflet  
  data/           # wards/, demographics/, fixtures/  
  scripts/        # dev and demo scripts  
  tools/          # PDF generator
```

22 Scaffold Routes

Backend endpoints already wired in the scaffold.

Method	Route	Service Called	Tables Touched
GET	/api/health	Health	—
GET	/api/version	Version	—
GET	/api/wards	ListWards	ward_boundaries
GET	/api/wards/{id}	GetWard	ward_boundaries, ward_demographics

GET	/api/wards/{id}/weather	GetWardWeather	ward_weather
GET	/api/wards/{id}/thermal	GetWardThermal	ward_thermal_stress
GET	/api/wards/{id}/risk	GetWardRisk	ward_mortality_risk
GET	/api/forecast	GetForecast	ward_thermal_stress, ward_mortality_risk
GET	/api/alerts	ListAlerts	alerts
POST	/api/agent/action-plan	AgentActionPlan	agent_decisions, alerts
POST	/api/pipeline/run	RunOnce	all

23

Environment Configuration

APP_MODE controls LIVE vs MOCK.

```
APP_MODE=MOCK
PORT=8080
OPEN_METEO_BASE=https://api.open-meteo.com/v1/forecast
HYDERABAD_LAT=17.3850
HYDERABAD_LON=78.4867
DB_URL=postgres://heatwave:heatwave@localhost:5432/heatwave
AGENT_LLM_PROVIDER=mock
LOG_LEVEL=info
AUTO_RUN=false
PIPELINE_INTERVAL=15m
```

24

Testing Strategy

Per-layer tests; deterministic fixtures for reproducibility.

Layer	Tests
Weather	API parse, unit conversion, missing-field handling
Spatial	Geometry validation, centroid, IDW correctness
Physics	Reference WBGT/UTCI cases, edge inputs
Risk	Determinism, band boundaries
Agent	Tool selection, no hallucination, schema validity
Backend	Route tests, integration tests, error codes
Frontend	Map rendering, API failure handling, ward selection

25

Failure Handling

Fail-safe fallbacks at every layer.

Failure	Impact	Detection	Fallback
Open-Meteo unavailable	No new forecast	HTTP timeout	Last good response
Missing radiation	WBGT degraded	Validation	Simplified WBGT
Invalid geometry	Spatial failure	Geometry check	Exclude + log
ML failure	No MRI	Inference error	Thermal-only status
Agent failure	No recommendation	LLM timeout	Rule-based action plan
Backend failure	Dashboard unavailable	Health check	Retry / cached page

26

Observability

Every log line is auditable.

Every log line includes: ts, level, service, pipeline_run_id, ward_id, stage, latency_ms, physics_version, risk_model_version. Reproducibility matters because alerts must be auditable.

27

Security & Safety

What this system must never do.

Hard Limits

No clinical diagnosis. No individual-level prediction. No fabricated mortality. All numeric outputs include provenance (physics_version, risk_model_version). Agent never overwrites scientific outputs. Banner: decision-support prototype.

28

24-Hour Execution Plan

Parallel streams; integration begins at hour 14.

Hour	Stream A (ML)	Stream B (Backend)	Stream C (Frontend)	Stream D (Agent)
0–2	Contracts + fixtures	Repo + DB	Repo + UI shell	Repo + tools
2–5	Weather + spatial	Routes + schema	Map base	Tool schemas
5–8	WBGT + UTCI	Orchestration	API client	Tool impl
8–11	Risk model	Persist + serve	Ward detail	Prompt + LLM
11–14	Vulnerability	Alerts	Forecast timeline	Structured output
14–17	Integration	E2E API	Wiring	Action plan
17–20	Mock fixtures	Health checks	Polish	Rule-based fallback
20–22	Tests	Tests	Tests	Tests

29

Team Parallelization Plan

Independent streams; integration near the end.

Four streams run in parallel. Integration begins at hour 14. ML/Physics, Backend, Frontend, and Agent each own their domain contracts and may not modify another team's outputs.

30

Domain-Specific Agent Prompts

Copy-paste prompts for each teammate's coding agent.

Prompt A — ML + Physics Agent

You are the ML + Physics engineer for the Heatwave Early Warning hackathon. Own weather ingestion, spatial downscaling, WBGT, UTCI, risk model. Stack: Python 3.11, NumPy, pandas, geopandas, shapely, scikit-learn. Files you OWN: ml/**, data/fixtures/**. Files you may READ but NOT modify: backend/**, agent/**, docs/API_CONTRACTS.md. Contracts: ward_weather, ward_thermal_stress, ward_mortality_risk. Tasks: Open-Meteo fetch, unit normalization, ward centroid + stable UUID, Liljegren WBGT with fallback, UTCI polynomial, vulnerability features, deterministic MRI + band + top factors. Forbidden: calculating decisions, calling LLM APIs, modifying backend contracts.

Prompt B — Backend Agent

You are the Backend engineer. Own Go service, DB, pipeline orchestration, alerts. Stack: Go 1.22+, Gin or Echo. Files you OWN: backend/**. Tasks: scaffold module, migrations, routes per API_CONTRACTS.md, scheduler, persist all stages, MOCK mode, /api/health, /api/version. Forbidden: implementing WBGT/UTCI, implementing MRI, modifying agent prompts.

Prompt C — Frontend Agent

You are the Frontend engineer. Own dashboard, GIS map, charts, ward detail, alert center. Stack: React 18, Vite, TypeScript, react-leaflet, recharts. Files you OWN: frontend/**. Tasks: choropleth map, ward detail panel, forecast timeline, alert center, decision panel, consume backend API, disclaimer banner visible, show physics_version and risk_model_version. Forbidden: computing WBGT/UTCI on browser, caching raw weather > 10 minutes.

Prompt D — Decision Agent

You are the Decision Agent engineer. Own agent prompts, tools, schemas, workflows, action planning. Stack: Python 3.11, custom tool loop or LangGraph, OpenAI-compatible LLM. Files you OWN: agent/**. Contracts: output strictly follows schema in docs/AGENT_SPEC.md. Never modify scientific outputs. Tasks: implement all 8 tools, system prompt enforcing factual grounding + structured output, rule-based fallback, schema validation, agent_run_id on every decision. Forbidden: computing WBGT/UTCI, inventing weather, overriding risk model, claiming mortality certainty.

31

Demo Flow

3–5 minute hackathon walkthrough.

1. Open Hyderabad choropleth. Show ward risk colors.
2. Click a high-risk ward → show WBGT, UTCI, MRI, band, top factors.
3. Switch to forecast timeline → show 3–5 day persistence.
4. Click Ask Agent → query 'What should the municipality do?'
5. Agent retrieves context and returns structured action plan.
6. Show alert generation in Alert Center.
7. Switch to MOCK mode → identical results with no internet.

32

Competitive Differentiation

Six layers of intelligence, one decision-support system.

The system combines six layers of intelligence: weather, spatial, physiological, vulnerability, temporal, decision. Most existing systems stop at weather. We go to decision with explicit, grounded reasoning. The agent is grounded in deterministic science; the agent never invents.

Sense → Agentic loop grounded in physics, not free-form LLM prose.
Predict →
Reason → Act
→ Re-evaluate

33

Limitations

What this system is not.

Honest Limitations Hackathon prototype. Not clinically validated. No ground-truth mortality dataset. Open-Meteo ~9 km resolution may under-resolve UHI. No real-time weather station telemetry. Agent decisions are advisory, not authoritative. MVP is read-only.

34

Traceability Matrix

Requirement → component → implementation → contract → demo.

Requirement	Component	Implementation	Contract	Demo Evidence
Fetch weather	ml/ingestion	open_meteo.py	raw_weather_staging	Live fetch
Downscale to ward	ml/spatial	wards.py	ward_weather	Map colors
Compute WBGT	ml/physics	wbgt.py	ward_thermal_stress	Ward detail

Compute UTCI	ml/physics	utci.py	ward_thermal_stress	Ward detail
Vulnerability features	ml/risk	risk_model.py	ward_demographics	Top factors
MRI	ml/risk	risk_model.py	ward_mortality_risk	Map + score
Decision	agent	agent.py	decision_plan	Decision panel
Alert	backend	alerts	alerts table	Alert center
Map	frontend	Map.tsx	/api/wards	Live demo
Disclaimer	frontend	Disclaimer.tsx	static	Visible banner

35

Definition of Done

MVP is complete only when every item is checked.

- [] Hyderabad ward polygons load
- [] Stable ward IDs exist
- [] Open-Meteo data is fetched
- [] Weather units are normalized
- [] Ward-level weather is generated
- [] WBGT calculated (Liljegren with fallback)
- [] UTCI calculated
- [] 3–5 day forecast exists
- [] Vulnerability features exist
- [] MRI generated
- [] Risk bands exist
- [] Agent retrieves risk
- [] Agent retrieves forecast
- [] Agent explains risk
- [] Agent generates structured actions
- [] Backend exposes APIs
- [] Frontend renders map
- [] Ward details work
- [] Alerts demonstrated
- [] MOCK mode works
- [] LIVE mode works
- [] End-to-end demo runs under 5 minutes

Closing Principle

Physics determines thermal stress. Risk estimates population-level risk. Agent reasons about decisions. Backend orchestrates. Frontend communicates.